

1. Compare Linear Regression and Polynomial Regression for predicting housing prices. Analyze differences in model complexity, ability to capture non-linear relationships, risk of overfitting, and interpretability. Discuss scenarios where each model performs better.

Q1. Linear Regression vs Polynomial Regression

	Feature Linear Regression	Polynomial Regression	Basic idea
	Fits a straight line	Fits a curved relationship	Equation $y=b_0+b_1x$ $y=b_0+b_1x+b_2x^2+\dots+b_nx^n$ Model
	complexity Low	Higher	
	Non-linear relationships	Cannot capture complex curves well	Can capture non-linear relationships
	Overfitting risk Low	Higher, especially with high degree	Interpretability Very easy More difficult
	Training time Fast	Generally slower as degree increases	
	Housing prices Good when price changes approximately linearly	Better when price has a curved relationship with features	
	Data requirement Works well with smaller datasets	Usually benefits from sufficient data like: House Area $\rightarrow$ Price	
	Example Price increases steadily with area		

Simple Example

Suppose housing data looks approximately

2000 sq.ft $\rightarrow$ ₹80 lakh

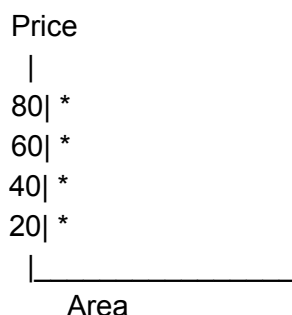
500 sq.ft $\rightarrow$ ₹20 lakh

1000 sq.ft $\rightarrow$ ₹40 lakh

1500 sq.ft $\rightarrow$ ₹60 lakh

Price increases differently at different area ranges

This is approximately linear:



Linear Regression works well here.

But suppose the price changes like:

500 sq.ft → ₹15 lakh

1000 sq.ft → ₹30 lakh

1500 sq.ft → ₹55 lakh

2000 sq.ft → ₹95 lakh

The relationship is curved. **Polynomial Regression** can model this better.

Overfitting

A major disadvantage of Polynomial Regression is that increasing the polynomial degree too much can make the model fit the training data extremely closely.

For example:

Degree 1 → Simple

Degree 2 → More flexible

Degree 3 → More flexible

Degree 10 → High risk of overfitting

When to use each?

Linear Regression:

- Relationship is approximately straight.
- Simple interpretation is important.
- Dataset is relatively simple.
- Fast prediction is required.

Polynomial Regression:

- Relationship between variables is curved.
- Linear Regression gives poor predictions.
- We need to capture non-linear patterns.

Conclusion

Linear Regression is simpler, faster, and easier to interpret, while Polynomial Regression is more flexible and capable of capturing non-linear relationships, but has a greater risk of overfitting.

2. Compare Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks for time-series prediction tasks such as stock price forecasting. Analyze differences in handling long-term dependencies, vanishing gradient

problem, computational complexity, and prediction accuracy.

The second question specifically asks for comparison in **long-term dependencies, vanishing gradients, computational complexity, and prediction accuracy.**

Feature RNN LSTM

Full form Recurrent Neural Network Long Short-Term Memory Memory Short-term memory

Short-term + long-term memory Architecture Simple recurrent structure Uses memory cell

and gates Long-term dependencies Difficult Handles them much better Vanishing gradient

More affected Designed to reduce this problem Computational complexity Lower Higher

Training speed Generally faster Generally slower Parameters Fewer More

Accuracy on long sequences Can decrease Usually better

Architecture Simple More complex

Better suited to longer dependencies

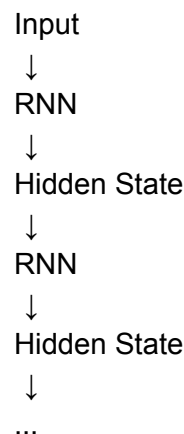
Stock prediction Can work for short patterns

Why does RNN have a problem?

Suppose stock prices are:

Day 1 → Day 2 → Day 3 → ... → Day 100

An RNN processes information sequentially:



When the sequence becomes very long, information from the early part may gradually become less influential.

This is related to the **vanishing gradient problem.**

How LSTM solves this

LSTM introduces a special memory mechanism called a **cell state** and uses gates.

The major gates are:

Forget Gate



Input Gate



Cell State



Output Gate

Forget Gate

Decides what old information should be removed.

Input Gate

Decides what new information should be stored.

Output Gate

Decides what information should be passed to the next stage. So

LSTM can preserve useful information for a much longer period.

Simple Example

Suppose we are predicting today's stock price.

Important information may include:

Yesterday's price

Last week's trend

Last month's trend

Previous market event

A basic RNN may have difficulty retaining very old information. LSTM is

designed to maintain important information over longer sequences.

Computational Complexity

RNN:

Simple architecture



Fewer parameters



Lower computation

LSTM:

Memory cell + gates



More parameters



More computation

So LSTM generally requires more computational resources than a basic RNN.

When to use?

RNN

Use when:

- Sequence is relatively short.
- Relationships are mostly recent.
- Computational simplicity is important.
- A basic sequential model is sufficient.

LSTM

Use when:

- Sequence is long.
- Long-term dependencies are important.
- Earlier information affects later predictions.
- Better handling of vanishing gradients is required.

Conclusion

RNNs are **simpler and computationally cheaper**, but they struggle with long-term dependencies. LSTMs are **more complex and computationally expensive**, but their memory

and gating mechanisms make them better suited for long sequences and long-term dependencies.

3. Compare Naïve Bayes and Support Vector Machines (SVM) for classification problems such as sentiment analysis. Analyze differences in assumptions, decision boundaries, computational efficiency, and performance on high-dimensional data. Discuss scenarios where each method is preferable.

The third question asks us to compare **Naïve Bayes and SVM** based on assumptions, decision boundaries, computational efficiency, high-dimensional data, and suitable applications.

Feature Naïve Bayes SVM

Type Probabilistic classifier Margin-based classifier Main principle Bayes'

theorem Maximum-margin separation

Assumption Features are conditionally independent

No such independence assumption

Based on probability Based on separating hyperplane

Decision boundary

Training Usually very fast Generally more computationally expensive

Prediction Very fast Fast after training

Performs well Performs very well

High-dimensional data

Text classification Excellent Excellent Small datasets Often

effective Can be effective Interpretability Relatively easy More

difficult

Probability output Natural probability estimates

Requires probability calibration for probabilities

Overfitting Can be relatively robust Depends on kernel and parameters

Naïve Bayes

Naïve Bayes uses Bayes' theorem:

$$P(C|X)=P(X)P(X|C)P(C)$$

where:

- C = class
- X = input features
- $P(C|X)$ = probability of class given input

For sentiment analysis:

"I love this movie"

↓

Naïve Bayes

↓

Positive

Another example:

"This movie is terrible"

↓

Naïve Bayes

↓

Negative

Why is it called "Naïve"?

It makes a strong assumption that features are conditionally independent.

For example:

"I really love this movie"

Naïve Bayes approximately treats the words as independent features when calculating the class probability.

This assumption is not always true, but the algorithm can still work very well, especially for text classification.

SVM

SVM tries to find a decision boundary that separates classes with the **maximum margin**.

For two classes:

Positive Positive

• •

• •

----- ← Decision Boundary

○ ○

○ ○

Negative Negative

The closest data points to the boundary are called **support vectors**.

Decision Boundary Difference

Naïve Bayes

Uses probabilities:

Features

↓

Calculate class probabilities

↓

Choose highest probability

SVM

Uses a separating boundary:

Training data

↓

Find maximum-margin boundary

↓

Classify new data

Computational Efficiency

Naïve Bayes

Training is usually very fast.

Large text dataset

↓

Naïve Bayes

↓

Fast training

This makes it useful when a large amount of text needs to be processed quickly.

SVM

SVM can require more computational resources during training, especially with large datasets or complex kernels.

However, once trained, prediction can be efficient.

High-Dimensional Data

Sentiment analysis can have thousands or even millions of possible word features.

For example:

Feature 1 → good

Feature 2 → bad

Feature 3 → excellent

Feature 4 → terrible

Feature 5 → movie

...

Feature 10,000 → word

Both Naïve Bayes and SVM can perform well on high-dimensional text data.

When to use Naïve Bayes?

Use Naïve Bayes when:

- Training speed is important.
- Dataset contains many text features.
- A simple model is preferred.
- Fast baseline classification is required.
- Probabilistic classification is useful.

Example

Email



Naïve Bayes



Spam / Not Spam

When to use SVM?

Use SVM when:

- A strong classification boundary is required.
- The dataset is high-dimensional.
- Clear class separation exists.
- More computational resources are available.
- Accuracy is more important than model simplicity.

Example

Customer Review



SVM



Positive / Negative



Final Comparison to Memorize

Linear Regression vs Polynomial Regression

Linear Regression



Straight-line relationship



Simple + interpretable

Polynomial Regression



Curved relationship



More flexible + higher overfitting risk

RNN vs LSTM

RNN



Simple



Short-term dependencies

↓
Vanishing gradient problem
LSTM
↓
Memory + Gates
↓
Long-term dependencies
↓
Better for long sequences

Naïve Bayes vs SVM

Naïve Bayes
↓
Probability-based
↓
Fast
↓
Excellent for text
SVM
↓
Maximum-margin boundary
↓
More computationally demanding
↓
Excellent for high-dimensional classification

One-line answers for viva

Question One-line answer Why use Polynomial

Regression? To model non-linear relationships.

Main disadvantage of Polynomial Regression? risk.

Why is LSTM better than RNN for long sequences? LSTM's memory cells and gates help preserve long-term information.
Higher model complexity and overfitting

What problem does LSTM address? Mainly the vanishing-gradient problem in long sequences.

Why is Naïve Bayes called naïve? It assumes conditional independence between features.

What does SVM maximize? The margin between classes. What are support vectors? Data points closest to the decision boundary.

Which is generally faster to train, Naïve Bayes or SVM?

Which is more complex, RNN or LSTM?

Which regression model is easier to interpret?
Naïve Bayes.

LSTM.

Linear Regression.